

PE-EC702A

Adaptive Signal Processing

Combined Module Notes

Modules 1 – 5

Semester VII • 3L:0T:0P • 3 Credits

CGEC | B.Tech ECE (2023–27)

Biraj Sarkar

Disclaimer / Study Guide Notice:

These are condensed **Short Notes** focusing on core theory, key formulas, block/circuit diagrams, and standard viva Q&As for university exam revision. Exhaustive numerical practice problems and derivation edge cases are not covered. This serves as a quick-revision reference guide for students.

Contents

Module 1: General Concept of Adaptive Filtering and Estimation	4
1 General Concept of Adaptive Filtering and Estimation	4
1.1 Basic Principles of Operation	4
1.2 Adaptive Filter Block Diagram	4
2 Applications and Motivation	5
2.1 The Four Classes of Adaptive Filter Applications	5
3 Review of Probability, Random Variables and Stationary Processes	6
3.1 Random Variables and Probability Density Functions	6
3.2 Stationary and Ergodicity	6
4 Correlation Structures and Correlation Matrices	6
4.1 The Autocorrelation Matrix	7
4.2 Properties of the Correlation Matrix	7
Module 2: Optimal FIR (Wiener) Filter	11
1 Optimal FIR (Wiener) Filter	11
1.1 Derivation of the Wiener-Hopf Equations	11
1.2 Minimum Mean-Square Error ($J_{\min}$)	11
2 Method of Steepest Descent	11
2.1 Weight Update Equation	12
2.2 Convergence Analysis and Stability	12
3 The Least Mean Squares (LMS) Algorithm	12
3.1 LMS Algorithms Formulations	13
3.2 LMS Convergence Analysis	13
3.3 Excess MSE and Misadjustment	13
Module 3: Variants of the LMS Algorithm	17
1 Variants of the LMS Algorithm	17
1.1 The Sign-LMS Family	17
1.2 Normalized LMS (NLMS) Algorithm	17
1.3 Block LMS (BLMS) and FFT-Based Realizations	17
1.4 Sub-band Adaptive Filtering	18
2 Signal Space Concepts	18
2.1 Vector Space and Subspace Theory	18
2.2 Inner Product Space and Orthogonality	18
2.3 Gram-Schmidt Orthogonalization	19
2.4 Orthogonal Projection	19

Module 4: Vector Space of Random Variables	23
1 Vector Space of Random Variables	23
2 Forward and Backward Projections	23
2.1 Forward Linear Prediction	23
2.2 Backward Linear Prediction	23
3 Stochastic Lattice Filters	24
3.1 Lattice Recursions	24
3.2 Optimal Reflection Coefficients	24
3.3 Single-Stage Lattice Block Diagram	24
4 Relationship with AR Modeling and Levinson-Durbin Recursion	25
4.1 Levinson-Durbin Recursion	25
5 Joint Process Estimator and Gradient Adaptive Lattice	25
5.1 Joint Process Estimator (JPE)	25
5.2 Gradient Adaptive Lattice (GAL)	25
Module 5: Introduction to Recursive Least Squares (RLS)	30
1 Introduction to Recursive Least Squares (RLS)	30
1.1 RLS Cost Function	30
1.2 Least-Squares Normal Equations	30
2 Vector Space Formulation of RLS and Matrix Inversion Lemma	30
2.1 Matrix Inversion Lemma (Woodbury Identity)	30
2.2 The Gain Vector	31
3 RLS Transversal Adaptive Filter Algorithm	31
3.1 RLS Lattice Filters	31
4 Advanced Topics	32
4.1 Affine Projection Algorithm (APA)	32
4.2 Subspace-Based Adaptive Filters	32
4.3 Partial Update Algorithms	32
4.4 QR Decomposition (QRD-RLS) and Systolic Arrays	32

Module 1

General Concept of Adaptive Filtering and Estimation

1. General Concept of Adaptive Filtering and Estimation

An adaptive filter is a self-designing, time-varying system that uses a recursive algorithm to adjust its parameters (coefficients) in order to minimize a predetermined cost function. Unlike fixed linear filters (e.g., standard FIR or IIR filters designed with static cutoff frequencies), adaptive filters automatically adjust to changes in the environment or statistics of the input signals.

Adaptive Filter: A digital filter whose transfer function is automatically updated by an adaptive algorithm in response to an incoming error signal, which is computed as the difference between a desired response and the filter's actual output.

1.1 Basic Principles of Operation

The operations of a typical adaptive filter are structured into two distinct stages:

- 1. Filtering Process:** The filter processes an input signal $x(n)$ to produce an output signal $y(n)$. For an M -tap transversal (FIR) filter, this is represented by:

$$y(n) = \sum_{k=0}^{M-1} w_k(n)x(n-k) = \mathbf{w}^T(n)\mathbf{x}(n) \quad (1)$$

where $\mathbf{w}(n) = [w_0(n), w_1(n), \dots, w_{M-1}(n)]^T$ is the weight vector, and $\mathbf{x}(n) = [x(n), x(n-1), \dots, x(n-M+1)]^T$ is the input vector.

- 2. Adaptive Process (Feedback):** An error signal $e(n)$ is generated by comparing the filter output $y(n)$ with a desired response $d(n)$:

$$e(n) = d(n) - y(n) \quad (2)$$

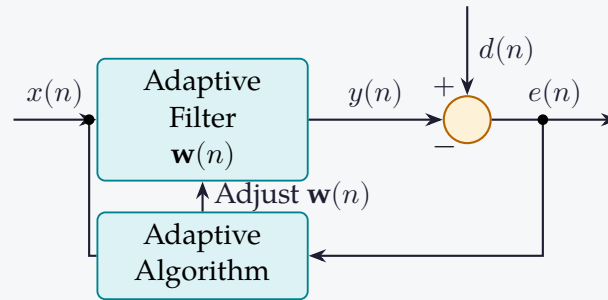
The adaptive algorithm (e.g., LMS or RLS) uses this error $e(n)$ to update the filter coefficients for the next time index:

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \Delta\mathbf{w}(n) \quad (3)$$

1.2 Adaptive Filter Block Diagram

The schematic block diagram below represents the closed-loop feedback structure of an adaptive filter:

Adaptive Filter Closed-Loop Block Diagram



2. Applications and Motivation

The primary motivation for adaptive filtering is the inability of classical filter designs to operate in environments where the signal characteristics are either unknown or time-varying. Typical applications are categorized into four basic classes depending on the choice of input $x(n)$ and desired response $d(n)$:

2.1 The Four Classes of Adaptive Filter Applications

1. **System Identification:** The adaptive filter models the transfer function of an unknown system.
 - *Setup:* Input $x(n)$ is fed to both the unknown system and the adaptive filter. The system output is the desired response $d(n)$.
 - *Application:* Echo cancellation in long-distance telephony, channel characterization.
2. **Inverse Modeling (Equalization):** The filter is placed in cascade with an unknown channel to compensate for channel distortion.
 - *Setup:* A known training signal is sent. The distorted channel output is input $x(n)$, and the original training signal is the desired response $d(n)$.
 - *Application:* High-speed digital communication channel equalization.
3. **Noise Cancellation (Interference Cancellation):** The filter estimates and subtracts noise from a corrupted signal.
 - *Setup:* Input $x(n)$ is a reference noise source. The desired response $d(n)$ is the signal corrupted by noise.
 - *Application:* Active noise control, cancellation of maternal ECG in fetal monitoring.
4. **Linear Prediction:** The filter predicts future values of a random process.
 - *Setup:* Input $x(n)$ is a delayed version of the signal, and the desired response $d(n)$ is the current signal value.
 - *Application:* Linear Predictive Coding (LPC) in speech compression, ADPCM encoders.

Application Class	Input Signal $x(n)$	Desired Signal $d(n)$	Objective
System ID	Reference excitation	Unknown system output	Find model $\mathbf{w}(n) \approx \text{system}$
Inverse Model	Distorted signal	Original source signal	Recover original source
Noise Cancellation	Noise reference source	Corrupted signal	Extract clean signal $e(n)$
Prediction	Delayed signal $s(n - \Delta)$	Current signal $s(n)$	Predict future signal values

3. Review of Probability, Random Variables and Stationary Processes

Adaptive filter design relies heavily on statistical signal processing. This section reviews the prerequisite concepts of random signals.

3.1 Random Variables and Probability Density Functions

A random variable X is a mapping from sample space to real numbers. It is described by its Probability Density Function (PDF) $f_X(x)$.

- **Expectation (Mean):** The statistical average of X :

$$\mu_X = \mathbb{E}[X] = \int_{-\infty}^{\infty} x f_X(x) dx \quad (4)$$

- **Variance:** The measure of signal spread/power:

$$\sigma_X^2 = \mathbb{E}[(X - \mu_X)^2] = \mathbb{E}[X^2] - \mu_X^2 \quad (5)$$

3.2 Stationary and Ergodicity

A random process $X(n)$ is a collection of random variables indexed by time.

- **Strict-Sense Stationarity (SSS):** A process is SSS if its joint probability distributions are invariant to any shift in the time origin.
- **Wide-Sense Stationarity (WSS):** A process is WSS if:

- Its mean is constant over time: $\mathbb{E}[X(n)] = \mu$
- Its autocorrelation depends only on the time difference (lag) k :

$$R_x(k) = \mathbb{E}[X(n)X^*(n - k)] \quad (6)$$

- **Ergodicity:** A WSS process is ergodic if its time averages equal its ensemble averages. This allows computing statistical metrics from a single, long time realization.

4. Correlation Structures and Correlation Matrices

Let $\mathbf{x}(n) = [x(n), x(n - 1), \dots, x(n - M + 1)]^T$ be an $M \times 1$ tap-input vector of a WSS process. The statistical relationship among its elements is captured by the autocorrelation matrix $\mathbf{R}$.

4.1 The Autocorrelation Matrix

The autocorrelation matrix $\mathbf{R}$ is defined as:

$$\mathbf{R} = \mathbb{E}[\mathbf{x}(n)\mathbf{x}^H(n)] \quad (7)$$

where H denotes the Hermitian transpose (conjugate transpose). In expanded form:

$$\mathbf{R} = \begin{bmatrix} R_x(0) & R_x(1) & \dots & R_x(M-1) \\ R_x(-1) & R_x(0) & \dots & R_x(M-2) \\ \vdots & \vdots & \ddots & \vdots \\ R_x(-M+1) & R_x(-M+2) & \dots & R_x(0) \end{bmatrix} \quad (8)$$

Since the process is WSS, $R_x(-k) = R_x^*(k)$.

4.2 Properties of the Correlation Matrix

The matrix $\mathbf{R}$ possesses several fundamental properties that dictate the behavior of adaptive filters:

1. **Hermitian Symmetry:** $\mathbf{R}$ is Hermitian, i.e., $\mathbf{R}^H = \mathbf{R}$.
2. **Toeplitz Structure:** All elements along any main diagonal are identical. This symmetry reduces computational cost for matrix inversion algorithms.
3. **Positive Semi-Definiteness:** For any non-zero complex vector $\mathbf{a}$:

$$\mathbf{a}^H \mathbf{R} \mathbf{a} = \mathbf{a}^H \mathbb{E}[\mathbf{x}(n)\mathbf{x}^H(n)] \mathbf{a} = \mathbb{E}[|\mathbf{a}^H \mathbf{x}(n)|^2] \geq 0 \quad (9)$$

If no component of the input signal is perfectly predictable, $\mathbf{R}$ is positive definite ($\mathbf{a}^H \mathbf{R} \mathbf{a} > 0$), meaning all its eigenvalues are strictly positive ($\lambda_i > 0$), ensuring $\mathbf{R}$ is invertible.

4. **Eigenvalue Bounds:** The eigenvalues of $\mathbf{R}$ are bounded by the minimum and maximum values of the Power Spectral Density (PSD) of the input process:

$$S_{\min} \leq \lambda_i \leq S_{\max} \quad (10)$$

Example: Correlation Matrix of a Sinusoid with Noise

Consider an input signal $x(n) = A \cos(\omega_0 n + \theta) + v(n)$, where θ is uniformly distributed in $[-\pi, \pi]$ and $v(n)$ is zero-mean white noise with variance σ_v^2 uncorrelated with the sinusoid. Compute the autocorrelation matrix for $M = 2$.

Solution: The autocorrelation function of the sinusoid is $R_s(k) = \frac{A^2}{2} \cos(\omega_0 k)$. The autocorrelation function of the white noise is $R_v(k) = \sigma_v^2 \delta(k)$. Since they are uncorrelated, the total autocorrelation is:

$$R_x(k) = R_s(k) + R_v(k) = \frac{A^2}{2} \cos(\omega_0 k) + \sigma_v^2 \delta(k)$$

For $k = 0$ and $k = 1$:

$$R_x(0) = \frac{A^2}{2} + \sigma_v^2$$

$$R_x(1) = \frac{A^2}{2} \cos(\omega_0)$$

Thus, the 2×2 correlation matrix is:

$$\mathbf{R} = \begin{bmatrix} \frac{A^2}{2} + \sigma_v^2 & \frac{A^2}{2} \cos(\omega_0) \\ \frac{A^2}{2} \cos(\omega_0) & \frac{A^2}{2} + \sigma_v^2 \end{bmatrix}$$

Quick Revision — Module I

Adaptive Filtering Concepts & Random Processes | PE-EC702A

Key Formulas at a Glance

Filter Output (FIR)	$y(n) = \mathbf{w}^T(n)\mathbf{x}(n) = \sum_{k=0}^{M-1} w_k(n)x(n-k)$
Error Signal	$e(n) = d(n) - y(n)$
Autocorrelation Function	$R_x(k) = \mathbb{E}[x(n)x^*(n-k)]$
Correlation Matrix	$\mathbf{R} = \mathbb{E}[\mathbf{x}(n)\mathbf{x}^H(n)]$
Cross-Correlation Vector	$\mathbf{p} = \mathbb{E}[\mathbf{x}(n)d^*(n)]$

One-Line Definitions

- **Adaptive Filter:** A digital filter that dynamically self-adjusts its coefficients to minimize a cost function.
- **Desired Response:** The target reference signal that the adaptive filter output attempts to replicate.
- **Wide-Sense Stationarity:** A process with constant mean and an autocorrelation depending only on lag.
- **Ergodicity:** A property where time-average statistics are identical to ensemble-average statistics.
- **Toeplitz Matrix:** A matrix where elements on each diagonal are constant, characteristic of WSS correlation matrices.
- **Hermitian Matrix:** A complex matrix equal to its own conjugate transpose, $\mathbf{R}^H = \mathbf{R}$.

Frequently Asked / Viva Questions

Q1. What is the primary difference between a fixed filter and an adaptive filter?

Fixed filters have static coefficients designed based on prior knowledge of signal spectra, whereas adaptive filters continuously update their coefficients in real-time based on statistical changes in the input signals.

Q2. Explain the role of the desired response $d(n)$ in an adaptive system.

The desired response acts as a target reference signal. The difference between $d(n)$ and the filter output $y(n)$ produces the error $e(n)$, which guides the optimization algorithm.

Q3. Identify the four main application classes of adaptive filters.

System Identification, Inverse Modeling (Equalization), Noise Cancellation, and Linear Prediction.

Frequently Asked / Viva Questions (contd.)

Q4. Why does the autocorrelation matrix $\mathbf{R}$ of a WSS process possess Toeplitz structure?

Because for a WSS process, the autocorrelation between $x(i)$ and $x(j)$ depends strictly on the lag $|i - j|$, causing elements along any diagonal to be identical.

Q5. State the significance of the positive definite property of a correlation matrix.

Positive definiteness implies that all eigenvalues of $\mathbf{R}$ are strictly positive ($\lambda_i > 0$), which guarantees that the matrix is non-singular and hence invertible.

Q6. What is ensemble average vs time average?

Ensemble average is the average across all possible realizations of a random variable at a specific time, while time average is computed along a single realization over infinite time.

Q7. Define a Hermitian matrix in the context of adaptive filtering.

A matrix $\mathbf{R}$ is Hermitian if $\mathbf{R}^H = \mathbf{R}$ (where H denotes the conjugate transpose). The autocorrelation matrix of any complex random process is always Hermitian.

Q8. What is the physical interpretation of $R_x(0)$?

$R_x(0) = \mathbb{E}[|x(n)|^2]$, which represents the average total power of the input signal $x(n)$.

Q9. What happens to the eigenvalues of $\mathbf{R}$ if the input is white noise?

All eigenvalues of the correlation matrix of white noise are identical and equal to the noise variance ($\lambda_i = \sigma^2$).

Q10. What is the cross-correlation vector $\mathbf{p}$?

It is the statistical correlation vector between the input signal vector $\mathbf{x}(n)$ and the desired response $d(n)$, defined as $\mathbf{p} = \mathbb{E}[\mathbf{x}(n)d^*(n)]$.

Comparison of Adaptive Filter Application Classes

Parameter	System ID	Noise Cancellation	Equalization
Filter Input $x(n)$	Excitation signal	Reference noise source	Distorted channel output
Desired Input $d(n)$	Output of unknown system	Corrupted signal	Known training signal
Desired Output	Model of system parameters	Cleaned signal $e(n)$	Reconstructed original data
Key Application	Echo cancellation in telephony	Active noise control, EEG	Wireless communications

Module 2

Optimal FIR (Wiener) Filter

1. Optimal FIR (Wiener) Filter

The Wiener filter is the optimal linear filter in the Mean-Square Error (MSE) sense. It assumes the signals are wide-sense stationary with known second-order statistics (autocorrelation and cross-correlation).

1.1 Derivation of the Wiener-Hopf Equations

Let the input vector be $\mathbf{x}(n) = [x(n), x(n-1), \dots, x(n-M+1)]^T$ and the weight vector be $\mathbf{w} = [w_0, w_1, \dots, w_{M-1}]^T$. The filter output is $y(n) = \mathbf{w}^H \mathbf{x}(n)$. The error signal is $e(n) = d(n) - y(n) = d(n) - \mathbf{w}^H \mathbf{x}(n)$. The cost function to minimize is the Mean-Square Error $J(\mathbf{w}) = \mathbb{E}[|e(n)|^2]$:

$$J(\mathbf{w}) = \mathbb{E}[(d(n) - \mathbf{w}^H \mathbf{x}(n))(d^*(n) - \mathbf{x}^H(n) \mathbf{w})] \quad (11)$$

Expanding the expectation:

$$\begin{aligned} J(\mathbf{w}) &= \mathbb{E}[|d(n)|^2] - \mathbf{w}^H \mathbb{E}[\mathbf{x}(n) d^*(n)] - \mathbb{E}[d(n) \mathbf{x}^H(n)] \mathbf{w} + \mathbf{w}^H \mathbb{E}[\mathbf{x}(n) \mathbf{x}^H(n)] \mathbf{w} \\ &= \sigma_d^2 - \mathbf{w}^H \mathbf{p} - \mathbf{p}^H \mathbf{w} + \mathbf{w}^H \mathbf{R} \mathbf{w} \end{aligned} \quad (12)$$

where $\sigma_d^2 = \mathbb{E}[|d(n)|^2]$, $\mathbf{p} = \mathbb{E}[\mathbf{x}(n) d^*(n)]$, and $\mathbf{R} = \mathbb{E}[\mathbf{x}(n) \mathbf{x}^H(n)]$. To find the optimal weight vector $\mathbf{w}_o$, we differentiate $J(\mathbf{w})$ with respect to the complex conjugate of the weight vector $\mathbf{w}^*$ (or compute the gradient) and set it to zero:

$$\nabla_{\mathbf{w}^*} J(\mathbf{w}) = -\mathbf{p} + \mathbf{R} \mathbf{w} = \mathbf{0} \quad (13)$$

This yields the **Wiener-Hopf Equations**:

$$\mathbf{R} \mathbf{w}_o = \mathbf{p} \implies \mathbf{w}_o = \mathbf{R}^{-1} \mathbf{p} \quad (14)$$

1.2 Minimum Mean-Square Error ($J_{\min}$)

Substituting the optimal weight $\mathbf{w}_o$ back into the cost function $J(\mathbf{w})$ gives the minimum mean-square error:

$$J_{\min} = \sigma_d^2 - \mathbf{w}_o^H \mathbf{p} = \sigma_d^2 - \mathbf{p}^H \mathbf{R}^{-1} \mathbf{p} \quad (15)$$

2. Method of Steepest Descent

When the statistics $\mathbf{R}$ and $\mathbf{p}$ are known, but inverting $\mathbf{R}$ is computationally expensive, we use the iterative Method of Steepest Descent. The weight vector is updated in the direction opposite to the gradient of the performance surface $J(\mathbf{w})$:

2.1 Weight Update Equation

$$\mathbf{w}(n+1) = \mathbf{w}(n) - \frac{1}{2}\mu\nabla J(\mathbf{w}(n)) \quad (16)$$

where μ is the step-size parameter. Since $\nabla J(\mathbf{w}) = -2\mathbf{p} + 2\mathbf{R}\mathbf{w}(n)$, we have:

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu(\mathbf{p} - \mathbf{R}\mathbf{w}(n)) \quad (17)$$

2.2 Convergence Analysis and Stability

Let the weight error vector be defined as $\mathbf{c}(n) = \mathbf{w}(n) - \mathbf{w}_o$. Subtracting $\mathbf{w}_o$ from both sides of the update equation:

$$\mathbf{c}(n+1) = (\mathbf{I} - \mu\mathbf{R})\mathbf{c}(n) \quad (18)$$

Applying the spectral decomposition $\mathbf{R} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^H$ and defining the rotated error vector $\mathbf{v}(n) = \mathbf{Q}^H\mathbf{c}(n)$, we get:

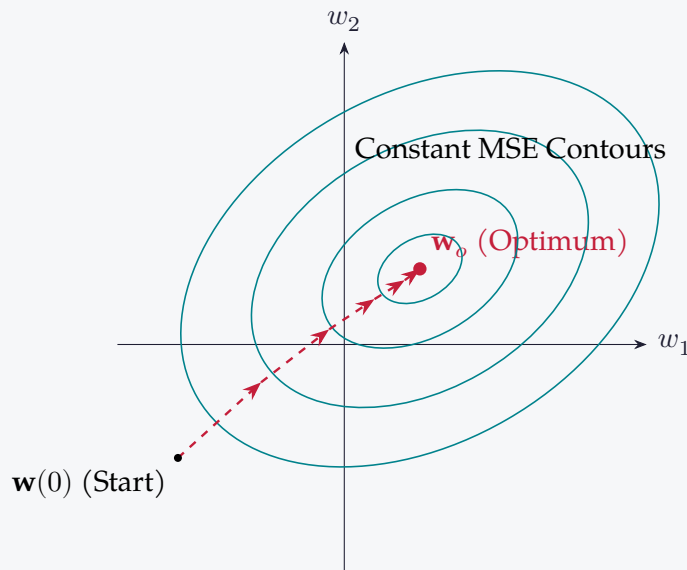
$$\mathbf{v}(n+1) = (\mathbf{I} - \mu\mathbf{\Lambda})\mathbf{v}(n) \quad (19)$$

For stability, each decoupled mode $v_i(n+1) = (1 - \mu\lambda_i)v_i(n)$ must decay to zero, which requires $|1 - \mu\lambda_i| < 1$ for all i :

$$0 < \mu < \frac{2}{\lambda_{\max}} \quad (20)$$

where $\lambda_{\max}$ is the largest eigenvalue of the correlation matrix $\mathbf{R}$.

Contour Lines of MSE Surface and Convergence Trajectory



3. The Least Mean Squares (LMS) Algorithm

The Least Mean Squares (LMS) algorithm is a stochastic gradient descent algorithm. It avoids the need to know or estimate the ensemble statistics $\mathbf{R}$ and $\mathbf{p}$ by using instantaneous approximations:

$$\hat{\mathbf{R}}(n) = \mathbf{x}(n)\mathbf{x}^H(n), \quad \hat{\mathbf{p}}(n) = \mathbf{x}(n)d^*(n) \quad (21)$$

3.1 LMS Algorithms Formulations

The instantaneous cost function is chosen as $\hat{J}(n) = |e(n)|^2$. The update equations are:

LMS Update Equations

Real-Valued LMS Algorithm:

$$e(n) = d(n) - \mathbf{w}^T(n)\mathbf{x}(n) \quad (22)$$

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu e(n)\mathbf{x}(n) \quad (23)$$

Complex-Valued LMS Algorithm:

$$e(n) = d(n) - \mathbf{w}^H(n)\mathbf{x}(n) \quad (24)$$

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu \mathbf{x}(n)e^*(n) \quad (25)$$

3.2 LMS Convergence Analysis

Under the **independence assumption** (the input vectors $\mathbf{x}(n)$ are statistically independent over time):

- **Convergence in the Mean:** The ensemble average of the weights converges to the Wiener solution, $\mathbb{E}[\mathbf{w}(n)] \rightarrow \mathbf{w}_o$, if:

$$0 < \mu < \frac{2}{\lambda_{\max}} \quad (26)$$

A conservative practical limit based on total signal power is:

$$0 < \mu < \frac{2}{\text{Tr}(\mathbf{R})} = \frac{2}{M \cdot R_x(0)} \quad (27)$$

- **Convergence in the Mean-Square:** For the second-order statistics of the weights to remain stable, the step-size must satisfy:

$$0 < \mu < \frac{2}{\sum_{i=1}^M \lambda_i} = \frac{2}{\text{Tr}(\mathbf{R})} \quad (28)$$

3.3 Excess MSE and Misadjustment

Due to gradient estimation noise, the LMS algorithm does not settle exactly at $J_{\min}$. It fluctuates about the optimum, introducing an **Excess Mean-Square Error** J_{ex} :

$$J_{\text{ex}} = \lim_{n \rightarrow \infty} \mathbb{E}[|e(n)|^2] - J_{\min} \quad (29)$$

For a small step-size μ , the excess MSE is given by:

$$J_{\text{ex}} \approx \frac{\mu J_{\min} \text{Tr}(\mathbf{R})}{2 - \mu \text{Tr}(\mathbf{R})} \quad (30)$$

The **Misadjustment** M is a dimensionless measure of how close the LMS filter is to the optimal Wiener filter:

$$M = \frac{J_{\text{ex}}}{J_{\min}} \approx \frac{\mu}{2} \text{Tr}(\mathbf{R}) = \frac{\mu}{2} \sum_{i=1}^M \lambda_i \quad (31)$$

Example: Wiener Filter Coefficient Design

A single-tap LMS filter has input signal $x(n)$ with variance $R_x(0) = 2$. The desired signal has variance $\sigma_d^2 = 5$, and the cross-correlation is $p = 1.6$.

1. Determine the optimal weight w_o .
2. Compute the minimum MSE $J_{\min}$.
3. Determine the maximum step-size μ for convergence in the mean.
4. Calculate the misadjustment M for $\mu = 0.05$.

Solution:

1. Using the Wiener-Hopf equation:

$$w_o = R^{-1}p = \frac{p}{R_x(0)} = \frac{1.6}{2} = 0.8$$

2. The minimum MSE $J_{\min}$ is:

$$J_{\min} = \sigma_d^2 - w_o p = 5 - (0.8 \times 1.6) = 5 - 1.28 = 3.72$$

3. For a single-tap filter ($M = 1$), $\lambda_{\max} = R_x(0) = 2$. The step-size range is:

$$0 < \mu < \frac{2}{\lambda_{\max}} \implies 0 < \mu < 1$$

4. The misadjustment M for $\mu = 0.05$:

$$M \approx \frac{\mu}{2} \text{Tr}(\mathbf{R}) = \frac{0.05}{2} \times 2 = 0.05 \text{ (or 5\%)}$$

Quick Revision — Module II

Wiener Filters, Steepest Descent & LMS | PE-EC702A

Key Formulas at a Glance

Wiener-Hopf Equation	$\mathbf{R}\mathbf{w}_o = \mathbf{p} \implies \mathbf{w}_o = \mathbf{R}^{-1}\mathbf{p}$
Minimum MSE	$J_{\min} = \sigma_d^2 - \mathbf{p}^H \mathbf{w}_o$
Steepest Descent Update	$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu(\mathbf{p} - \mathbf{R}\mathbf{w}(n))$
LMS Update (Complex)	$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu \mathbf{x}(n) e^*(n)$
Step-Size Bounds	$0 < \mu < \frac{2}{\lambda_{\max}}$ (Ensemble) or $0 < \mu < \frac{2}{\text{Tr}(\mathbf{R})}$ (LMS)
Misadjustment	$M = \frac{J_{\text{ex}}}{J_{\min}} \approx \frac{\mu}{2} \text{Tr}(\mathbf{R})$

One-Line Definitions

- **Wiener Filter:** The optimal linear time-invariant filter that minimizes mean-square error.
- **Steepest Descent:** An iterative optimization method that moves weights along the direction of maximum negative gradient.
- **LMS Algorithm:** A low-complexity adaptive algorithm using instantaneous signal values to approximate gradients.
- **Misadjustment:** A measure of the average excess MSE relative to the minimum MSE, representing algorithm noise.
- **Eigenvalue Spread:** The ratio $\lambda_{\max}/\lambda_{\min}$, indicating the convergence rate limit of gradient algorithms.
- **Independence Assumption:** An analytical simplification stating that successive tap-input vectors are statistically independent.

Frequently Asked / Viva Questions

Q1. Derive the Wiener-Hopf equations for a real FIR filter.

See Section 1.1: We expand $J(\mathbf{w}) = \mathbb{E}[(d(n) - \mathbf{w}^T \mathbf{x}(n))^2] = \sigma_d^2 - 2\mathbf{w}^T \mathbf{p} + \mathbf{w}^T \mathbf{R} \mathbf{w}$. Differentiating w.r.t. $\mathbf{w}$ and setting to $\mathbf{0}$ yields $\mathbf{R}\mathbf{w}_o = \mathbf{p}$.

Q2. What is the convergence criterion for the method of steepest descent?

The step-size parameter must satisfy $0 < \mu < 2/\lambda_{\max}$, where $\lambda_{\max}$ is the maximum eigenvalue of the autocorrelation matrix $\mathbf{R}$.

Q3. Why is LMS referred to as a "stochastic" gradient algorithm?

Because it replaces the true gradient (which requires expectations of signals)

Frequently Asked / Viva Questions (contd.)

with a noisy instantaneous gradient estimate computed directly from the current samples.

Q4. Write down the complex LMS weight update equation and explain each term.

$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu \mathbf{x}(n) e^*(n)$, where $\mathbf{w}(n)$ is the weight vector, μ is the step-size, $\mathbf{x}(n)$ is the input vector, and $e^*(n)$ is the complex conjugate of the error.

Q5. What is the eigenvalue spread and how does it affect convergence speed?

Eigenvalue spread is the ratio $\chi = \lambda_{\max}/\lambda_{\min}$. A large eigenvalue spread (highly correlated input) causes the MSE surface to form a narrow valley, resulting in slow convergence.

Q6. What is the physical meaning of misadjustment in LMS?

It represents the cost of adaptation; the weight vector continuously hunts around the optimum, causing the average MSE at steady state to be slightly larger than $J_{\min}$.

Q7. What is the conservative step-size limit used in practice for LMS?

$\mu < 2/\text{Tr}(\mathbf{R}) = 2/(M \cdot \text{Signal Power})$, which is easily calculated since it only requires the input signal power.

Q8. Can the LMS algorithm converge if the step-size μ is negative?

No, if $\mu < 0$, the algorithm moves in the direction of the positive gradient, leading to divergence (instability) and weights going to infinity.

Q9. How does the number of filter taps M affect the misadjustment?

Since $M \approx \text{Tr}(\mathbf{R})/P_x$, a larger number of filter taps increases $\text{Tr}(\mathbf{R})$, which increases the misadjustment for a fixed step-size μ .

Q10. What is the minimum MSE ($J_{\min}$) of a system where the input and desired signal are uncorrelated?

If they are uncorrelated, the cross-correlation vector $\mathbf{p} = \mathbf{0}$, giving $\mathbf{w}_o = \mathbf{0}$, which yields $J_{\min} = \sigma_d^2$ (the total power of the desired signal).

Module 3

Variants of the LMS Algorithm

1. Variants of the LMS Algorithm

To overcome constraints of standard LMS (computational complexity, time-varying power instability, or convergence rates), several variants are employed.

1.1 The Sign-LMS Family

Designed to reduce computational requirements in high-speed hardware by replacing multiplications with sign operations. The vector sign function $\text{sgn}(\cdot)$ extracts element-wise signs (± 1 or 0).

1. **Sign-Error LMS:** Simplifies the error term.

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu \text{sgn}(e(n))\mathbf{x}(n) \quad (32)$$

2. **Sign-Data (Regressor) LMS:** Simplifies the regressor data vector.

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu e(n) \text{sgn}(\mathbf{x}(n)) \quad (33)$$

3. **Sign-Sign LMS:** Simplifies both terms; updates require only additions (no multiplications).

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu \text{sgn}(e(n)) \text{sgn}(\mathbf{x}(n)) \quad (34)$$

1.2 Normalized LMS (NLMS) Algorithm

In standard LMS, the step-size is bounded by the signal power ($0 < \mu < 2/\text{Tr}(\mathbf{R})$). If the input power varies widely (like speech), standard LMS can diverge during high-power segments. NLMS solves this by normalizing the step-size with the instantaneous squared norm of the input vector:

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \frac{\tilde{\mu}}{\|\mathbf{x}(n)\|^2 + \epsilon} e^*(n) \mathbf{x}(n) \quad (35)$$

where $\tilde{\mu}$ is the normalized step-size ($0 < \tilde{\mu} < 2$), and $\epsilon > 0$ is a small regularization parameter preventing division by zero.

1.3 Block LMS (BLMS) and FFT-Based Realizations

Standard LMS updates weights sample-by-sample. Block LMS updates the weights only once per block of L input samples, averaging the gradients over the block:

$$\mathbf{w}(k+1) = \mathbf{w}(k) + \frac{2\mu}{L} \sum_{i=0}^{L-1} e(kL+i) \mathbf{x}(kL+i) \quad (36)$$

where k is the block index.

- **FFT Realization (FDAF):** Block convolution and correlation can be computed in the frequency domain using Fast Fourier Transforms (FFTs) via the overlap-save or overlap-add method. This reduces computational complexity from $O(M^2)$ to $O(M \log_2 M)$ for large tap lengths.

1.4 Sub-band Adaptive Filtering

In sub-band adaptive filters, the wideband input and desired signals are split into N sub-bands using an analysis filter bank. Adaptive filtering is performed on each sub-band signal after decimation (downsampling). Finally, a synthesis filter bank reconstructs the full-band output.

- **Advantages:** Decimation reduces the operating sampling rate. Sub-band signals have a much whiter spectral envelope (smaller eigenvalue spread) than the original wideband signal, significantly accelerating convergence.

2. Signal Space Concepts

To formulate advanced algorithms like RLS, signals are viewed as elements of abstract vector spaces (Hilbert spaces).

2.1 Vector Space and Subspace Theory

A vector space V over a field F (typically complex numbers $\mathbb{C}$) is a set of elements closed under vector addition and scalar multiplication, satisfying the standard vector space axioms.

- **Subspace:** A subset $W \subseteq V$ is a subspace if it is closed under vector addition and scalar multiplication.
- **Basis and Dimension:** A set of vectors $\{v_1, \dots, v_k\}$ forms a basis for V if they are linearly independent and span V . The number of vectors in the basis is the dimension $\dim(V)$.
- **Linear Operators, Rank and Nullity:** For a linear operator $T : V \rightarrow W$, the range space is $\mathcal{R}(T)$ and null space is $\mathcal{N}(T)$. The dimensions are rank $\rho(T) = \dim(\mathcal{R}(T))$ and nullity $\nu(T) = \dim(\mathcal{N}(T))$. The Rank-Nullity Theorem states:

$$\rho(T) + \nu(T) = \dim(V) \quad (37)$$

2.2 Inner Product Space and Orthogonality

An inner product space is a vector space V equipped with an inner product $\langle u, v \rangle$, mapping two vectors to a scalar, satisfying conjugate symmetry ($\langle u, v \rangle = \langle v, u \rangle^*$), linearity, and positive definiteness ($\langle u, u \rangle \geq 0$).

- **Induced Norm:** $\|u\| = \sqrt{\langle u, u \rangle}$.
- **Orthogonality:** Two vectors are orthogonal if $\langle u, v \rangle = 0$.
- **Orthogonal Decomposition:** Any vector space V with a closed subspace W can be decomposed uniquely as:

$$V = W \oplus W^\perp \quad (38)$$

where $W^\perp = \{v \in V \mid \langle v, w \rangle = 0 \ \forall w \in W\}$ is the orthogonal complement.

2.3 Gram-Schmidt Orthogonalization

A recursive algorithm that converts a set of linearly independent vectors $\{v_1, v_2, \dots, v_M\}$ into an orthonormal set $\{q_1, q_2, \dots, q_M\}$ spanning the same space:

$$\begin{aligned} u_1 &= v_1, \quad q_1 = \frac{u_1}{\|u_1\|} \\ u_k &= v_k - \sum_{j=1}^{k-1} \langle v_k, q_j \rangle q_j, \quad q_k = \frac{u_k}{\|u_k\|} \quad \text{for } k = 2, \dots, M \end{aligned} \quad (39)$$

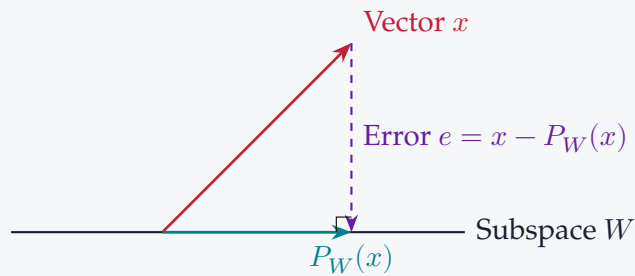
2.4 Orthogonal Projection

The orthogonal projection of a vector x onto a subspace W spanned by orthonormal vectors $\{q_1, \dots, q_k\}$ is:

$$P_W(x) = \sum_{i=1}^k \langle x, q_i \rangle q_i \quad (40)$$

The projection error $e = x - P_W(x)$ is orthogonal to the subspace W ($\langle e, w \rangle = 0 \quad \forall w \in W$), which is the geometric basis of the Orthogonality Principle.

Vector Projection Geometry



Example: Gram-Schmidt Orthonormalization

Apply the Gram-Schmidt process to the following set of vectors in $\mathbb{R}^3$ to obtain an orthonormal basis:

$$v_1 = [1, 1, 0]^T, \quad v_2 = [1, 2, 0]^T, \quad v_3 = [0, 1, 2]^T$$

Solution:

1. For v_1 :

$$u_1 = v_1 = [1, 1, 0]^T \Rightarrow \|u_1\| = \sqrt{1^2 + 1^2 + 0^2} = \sqrt{2}$$

$$q_1 = \frac{u_1}{\|u_1\|} = \left[\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}, 0 \right]^T$$

2. For v_2 :

$$\langle v_2, q_1 \rangle = (1 \times \frac{1}{\sqrt{2}}) + (2 \times \frac{1}{\sqrt{2}}) + 0 = \frac{3}{\sqrt{2}}$$

$$u_2 = v_2 - \langle v_2, q_1 \rangle q_1 = \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix} - \frac{3}{\sqrt{2}} \begin{bmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix} - \begin{bmatrix} 1.5 \\ 1.5 \\ 0 \end{bmatrix} = \begin{bmatrix} -0.5 \\ 0.5 \\ 0 \end{bmatrix}$$

$$\|u_2\| = \sqrt{(-0.5)^2 + 0.5^2 + 0} = \sqrt{0.5} = \frac{1}{\sqrt{2}}$$

$$q_2 = \frac{u_2}{\|u_2\|} = \sqrt{2} \begin{bmatrix} -0.5 \\ 0.5 \\ 0 \end{bmatrix} = \left[-\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}, 0 \right]^T$$

3. For v_3 :

$$\langle v_3, q_1 \rangle = \frac{1}{\sqrt{2}}, \quad \langle v_3, q_2 \rangle = \frac{1}{\sqrt{2}}$$

$$u_3 = v_3 - \langle v_3, q_1 \rangle q_1 - \langle v_3, q_2 \rangle q_2 = \begin{bmatrix} 0 \\ 1 \\ 2 \end{bmatrix} - \frac{1}{2} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} - \frac{1}{2} \begin{bmatrix} -1 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 2 \end{bmatrix}$$

$$\|u_3\| = 2 \Rightarrow q_3 = \frac{u_3}{\|u_3\|} = [0, 0, 1]^T$$

The orthonormal basis is:

$$\left\{ \begin{bmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \\ 0 \end{bmatrix}, \begin{bmatrix} -1/\sqrt{2} \\ 1/\sqrt{2} \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \right\}$$

Quick Revision — Module III

LMS Variants & Signal Space Concepts | PE-EC702A

Key Formulas at a Glance

Sign-Error LMS	$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu \operatorname{sgn}(e(n))\mathbf{x}(n)$
Sign-Sign LMS	$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu \operatorname{sgn}(e(n)) \operatorname{sgn}(\mathbf{x}(n))$
Normalized LMS Update	$\mathbf{w}(n+1) = \mathbf{w}(n) + \frac{\tilde{\mu}}{\ \mathbf{x}(n)\ ^2 + \epsilon} e^*(n) \mathbf{x}(n)$
Rank-Nullity Theorem	$\operatorname{rank}(T) + \operatorname{nullity}(T) = \dim(V)$
Orthogonal Projection	$P_W(x) = \sum_{i=1}^k \langle x, q_i \rangle q_i \quad (q_i: \text{ orthonormal basis})$
Gram-Schmidt Step	$u_k = v_k - \sum_{j=1}^{k-1} \langle v_k, q_j \rangle q_j, \quad q_k = u_k / \ u_k\ $

One-Line Definitions

- **Normalized LMS:** A variant of LMS that divides step-size by input power to maintain stability.
- **Sign-Sign LMS:** A hardware-friendly LMS variant using only signs of data and error.
- **Inner Product Space:** A vector space equipped with a function defining angles and lengths (norms).
- **Gram-Schmidt Process:** An algorithm generating orthonormal bases from a set of linearly independent vectors.
- **Rank of Operator:** The dimension of the range space of a linear transformation.
- **Nullity of Operator:** The dimension of the null space of a linear transformation.

Frequently Asked / Viva Questions

Q1. What is the primary advantage of Sign-LMS algorithms?

They reduce hardware multiplier requirements. Sign-Sign LMS replaces the update multiplications with simple addition/subtraction, accelerating hardware processing.

Q2. Why does standard LMS sometimes diverge with speech inputs and how does NLMS solve it?

Speech has highly variable power. During high-amplitude segments, the fixed step-size of standard LMS can violate bounds ($2/\operatorname{Tr}(\mathbf{R})$), causing divergence. NLMS scales μ inversely with instantaneous power, ensuring stability.

Q3. State the Rank-Nullity Theorem.

For any linear operator $T : V \rightarrow W$ where V is finite-dimensional, $\dim(\mathcal{R}(T)) +$

Frequently Asked / Viva Questions (contd.)

$\dim(\mathcal{N}(T)) = \dim(V)$, meaning Rank + Nullity = Dimension of domain.

Q4. Explain overlap-save block processing in FDAF.

It is a frequency-domain technique that computes block convolutions of input signals with filter weights using DFTs/FFT, avoiding time-domain convolution overhead.

Q5. What is sub-band adaptive filtering and why is it used?

It divides a wideband signal into multiple narrower frequency bands. It speeds up convergence by reducing the eigenvalue spread (whitening) and lowers computational complexity by decimation.

Q6. Define the orthogonal complement of a subspace.

The orthogonal complement $W^\perp$ of a subspace W is the set of all vectors in the vector space that are orthogonal to every vector in W .

Q7. Write the formula for the orthogonal projection of vector x onto vector y .

$$P_y(x) = \frac{\langle x, y \rangle}{\|y\|^2} y.$$

Q8. What represents the inner product of two signals $x(n)$ and $y(n)$ in L_2 space?

$\langle x, y \rangle = \sum_{n=-\infty}^{\infty} x(n)y^*(n)$ (for discrete signals) or $\int x(t)y^*(t)dt$ (for continuous signals).

Q9. What is the regularization parameter ϵ in NLMS?

It is a small positive constant added to the denominator to prevent division by zero (or numerical instability) when the input vector norm $\|\mathbf{x}(n)\|$ is zero or very small.

Q10. How does the computational complexity of FDAF scale with filter length M ?

It scales as $O(M \log_2 M)$ compared to $O(M^2)$ for standard time-domain LMS.

Module 4

Vector Space of Random Variables

1. Vector Space of Random Variables

Random variables can be modeled as vectors in a Hilbert space H .

- **Inner Product:** For zero-mean random variables X and Y , the inner product is defined as their correlation:

$$\langle X, Y \rangle = \mathbb{E}[XY^*] \quad (41)$$

- **Norm:** The norm corresponds to the root-mean-square (RMS) value:

$$\|X\| = \sqrt{\langle X, X \rangle} = \sqrt{\mathbb{E}[|X|^2]} \quad (42)$$

- **Orthogonality:** Two random variables are orthogonal if $\langle X, Y \rangle = \mathbb{E}[XY^*] = 0$, meaning they are statistically uncorrelated.

2. Forward and Backward Projections

Linear prediction involves estimating a signal sample from adjacent samples.

2.1 Forward Linear Prediction

Predicts the current sample $x(n)$ based on the past M samples $\{x(n-1), \dots, x(n-M)\}$.

- **Forward Prediction Error ($f_M(n)$):**

$$f_M(n) = x(n) - \sum_{k=1}^M a_{M,k}^* x(n-k) \quad (43)$$

where $a_{M,k}$ are the forward prediction coefficients.

2.2 Backward Linear Prediction

Predicts the past sample $x(n-M)$ using the current and subsequent M samples $\{x(n), \dots, x(n-M+1)\}$.

- **Backward Prediction Error ($g_M(n)$):**

$$g_M(n) = x(n - M) - \sum_{k=1}^M b_{M,k}^* x(n - k + 1) \quad (44)$$

where $b_{M,k}$ are the backward prediction coefficients. For WSS processes, $b_{M,k} = a_{M,M-k+1}^*$.

3. Stochastic Lattice Filters

Lattice filters represent a modular structure realizing both forward and backward prediction errors order-recursively.

3.1 Lattice Recursions

For a stage m (where $m = 1, 2, \dots, M$):

Lattice Prediction Error Updates

$$f_m(n) = f_{m-1}(n) - \kappa_m^* g_{m-1}(n-1) \quad (45)$$

$$g_m(n) = g_{m-1}(n-1) - \kappa_m f_{m-1}(n) \quad (46)$$

Initial conditions for the recursion are: $f_0(n) = g_0(n) = x(n)$.

Here, κ_m is the **Reflection Coefficient** (PARCOR coefficient) for stage m .

3.2 Optimal Reflection Coefficients

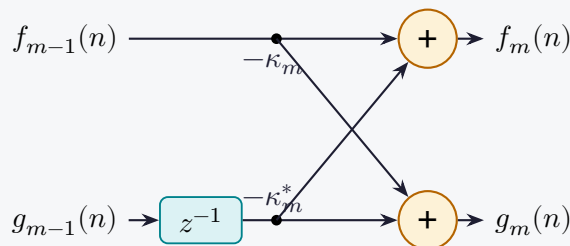
To minimize the sum of the forward and backward mean square errors ($\mathbb{E}[|f_m(n)|^2] + \mathbb{E}[|g_m(n)|^2]$), we differentiate with respect to κ_m , giving:

$$\kappa_m = \frac{2\mathbb{E}[f_{m-1}(n)g_{m-1}^*(n-1)]}{\mathbb{E}[|f_{m-1}(n)|^2] + \mathbb{E}[|g_{m-1}(n-1)|^2]} \quad (47)$$

By Cauchy-Schwarz, $|\kappa_m| \leq 1$. This ensures that the all-pole synthesis filter (inverse lattice filter) is guaranteed to be stable if and only if $|\kappa_m| < 1$ for all m .

3.3 Single-Stage Lattice Block Diagram

Lattice Filter Stage m Block Structure



4. Relationship with AR Modeling and Levinson-Durbin Recursion

An Autoregressive (AR) process of order M is generated by filtering white noise through an all-pole filter. The AR parameters and reflection coefficients are equivalent representations of the same process.

4.1 Levinson-Durbin Recursion

It solves the Yule-Walker equations $\mathbf{R}\mathbf{a} = \mathbf{r}$ with $O(M^2)$ complexity instead of $O(M^3)$ matrix inversion. The recursion steps for order $m = 1, 2, \dots, M$:

$$\Delta_{m-1} = r(m) - \sum_{i=1}^{m-1} a_{m-1,i} r(m-i) \quad (48)$$

$$\kappa_m = \frac{\Delta_{m-1}}{E_{m-1}} \quad (49)$$

$$a_{m,i} = a_{m-1,i} - \kappa_m a_{m-1,m-i}^* \quad (i = 1, \dots, m-1) \quad (50)$$

$$a_{m,m} = \kappa_m \quad (51)$$

$$E_m = E_{m-1}(1 - |\kappa_m|^2) \quad (52)$$

where $E_0 = r(0)$ is the input variance, and κ_m is the reflection coefficient.

5. Joint Process Estimator and Gradient Adaptive Lattice

5.1 Joint Process Estimator (JPE)

The JPE uses the backward prediction errors $g_m(n)$ of a lattice predictor as the input to a transversal filter to estimate a desired signal $d(n)$. Because the backward errors $\{g_0(n), \dots, g_M(n)\}$ are mutually orthogonal, the estimator weights h_m can be optimized independently:

$$\hat{d}(n) = \sum_{m=0}^M h_m^* g_m(n), \quad h_m = \frac{\mathbb{E}[d(n)g_m^*(n)]}{\mathbb{E}[|g_m(n)|^2]} \quad (53)$$

5.2 Gradient Adaptive Lattice (GAL)

GAL is the adaptive implementation of the lattice filter. The reflection coefficients κ_m are updated recursively using a stochastic gradient approach:

$$D_m(n) = \beta D_m(n-1) + (1 - \beta) [|f_m(n)|^2 + |g_m(n-1)|^2] \quad (54)$$

$$\kappa_{m+1}(n+1) = \kappa_{m+1}(n) + \frac{\mu}{D_m(n)} [f_m(n)g_m^*(n-1)] \quad (55)$$

where β is a smoothing factor and $D_m(n)$ estimates the power at stage m .

Example: Levinson-Durbin Recursion

Given autocorrelation values $r(0) = 1.0, r(1) = 0.5, r(2) = 0.125$. Run the Levinson-Durbin recursion to find the reflection coefficients κ_1, κ_2 and the predictor coefficients of a second-order predictor.

Solution: Initialize: $E_0 = r(0) = 1.0$. **Iteration 1** ($m = 1$):

$$\Delta_0 = r(1) = 0.5$$

$$\kappa_1 = \frac{\Delta_0}{E_0} = \frac{0.5}{1.0} = 0.5$$

Predictor coefficient:

$$a_{1,1} = \kappa_1 = 0.5$$

Update error energy:

$$E_1 = E_0(1 - \kappa_1^2) = 1.0(1 - 0.25) = 0.75$$

Iteration 2 ($m = 2$):

$$\Delta_1 = r(2) - a_{1,1}r(1) = 0.125 - (0.5 \times 0.5) = 0.125 - 0.25 = -0.125$$

$$\kappa_2 = \frac{\Delta_1}{E_1} = \frac{-0.125}{0.75} = -\frac{1}{6} \approx -0.1667$$

Predictor coefficients:

$$a_{2,2} = \kappa_2 = -0.1667$$

$$a_{2,1} = a_{1,1} - \kappa_2 a_{1,1} = 0.5 - \left(-\frac{1}{6} \times 0.5\right) = 0.5 + 0.0833 = 0.5833$$

Update error energy:

$$E_2 = E_1(1 - \kappa_2^2) = 0.75 \left(1 - \frac{1}{36}\right) = 0.75 \times \frac{35}{36} = 0.7292$$

The reflection coefficients are $\kappa_1 = 0.5, \kappa_2 = -0.1667$, and predictor coefficients are $a_{2,1} = 0.5833, a_{2,2} = -0.1667$.

Quick Revision — Module IV

Lattice Filters & Linear Prediction | PE-EC702A

Key Formulas at a Glance

Forward Error	$f_m(n) = f_{m-1}(n) - \kappa_m^* g_{m-1}(n-1)$
Backward Error	$g_m(n) = g_{m-1}(n-1) - \kappa_m f_{m-1}(n)$
Reflection Coeff. (κ_m)	$\kappa_m = \frac{2\mathbb{E}[f_{m-1}(n)g_{m-1}^*(n-1)]}{\mathbb{E}[f_{m-1}(n) ^2] + \mathbb{E}[g_{m-1}(n-1) ^2]}$
Levinson-Durbin κ_m	$\kappa_m = \Delta_{m-1}/E_{m-1}$
Predictor Coefficient Update	$a_{m,i} = a_{m-1,i} - \kappa_m a_{m-1,m-i}^* \quad (a_{m,m} = \kappa_m)$
Error Energy Update	$E_m = E_{m-1}(1 - \kappa_m ^2)$

One-Line Definitions

- **Forward Prediction Error:** The difference between the current signal sample and its linear estimate based on past samples.
- **Backward Prediction Error:** The difference between a past signal sample and its estimate based on subsequent samples.
- **Reflection Coefficient:** The correlation metric κ_m bounded in $[-1, 1]$ parameterizing a lattice stage.
- **Levinson-Durbin Recursion:** An efficient $O(M^2)$ recursive algorithm to solve the Yule-Walker equations.
- **Joint Process Estimator:** A structure utilizing orthogonal backward errors to estimate an external desired signal.
- **GAL Filter:** A gradient-based adaptive lattice filter that adjusts reflection coefficients stage-by-stage.

Frequently Asked / Viva Questions

Q1. What are the initial conditions for the lattice filter recursions?

$f_0(n) = g_0(n) = x(n)$, representing the raw input sample.

Q2. Explain why lattice filters are preferred in hardware over transversal (FIR) filters.

Lattice filters are modular. Increasing the filter order simply requires adding more identical stages at the end, without recalculating prior stages' coefficients.

Q3. What condition must reflection coefficients satisfy to ensure system stability?

$|\kappa_m| < 1$ for all stages m . If any $|\kappa_m| \geq 1$, the corresponding synthesis filter

Frequently Asked / Viva Questions (contd.)

becomes unstable.

Q4. Why are backward prediction errors $g_m(n)$ orthogonal to each other?

Because $g_m(n)$ represents the component of $x(n-m)$ that is orthogonal to the subspace spanned by $\{x(n), \dots, x(n-m+1)\}$. Thus, $\mathbb{E}[g_i(n)g_j^*(n)] = 0$ for $i \neq j$.

Q5. How does the computational complexity of Levinson-Durbin compare to direct matrix inversion?

Levinson-Durbin requires $O(M^2)$ operations, whereas direct inversion of the correlation matrix via Gaussian elimination requires $O(M^3)$ operations.

Q6. What is the relationship between predictor coefficients and reflection coefficients?

Predictor coefficients $a_{M,k}$ are calculated from reflection coefficients κ_m using the step-up recursion relation: $a_{m,i} = a_{m-1,i} - \kappa_m a_{m-1,m-i}^*$.

Q7. What is a Joint Process Estimator (JPE)?

It is an adaptive structure that first orthogonalizes the input signal using a lattice filter, and then estimates a desired response by forming a weighted linear combination of the orthogonal backward prediction errors.

Q8. Define the PARCOR coefficient.

It stands for PARTial CORrelation coefficient, which is mathematically identical to the reflection coefficient κ_m in a lattice filter.

Q9. Why does the error energy E_m decrease with increase in predictor order m ?

Because $E_m = E_{m-1}(1 - |\kappa_m|^2)$, and since $|\kappa_m| < 1$, the factor $(1 - |\kappa_m|^2) \leq 1$, meaning adding more taps can only reduce (or keep equal) the prediction error power.

Q10. What is the role of the power estimator $D_m(n)$ in the GAL algorithm?

It normalizes the step-size for each stage individually (similar to NLMS), ensuring that convergence speed is uniform across all stages regardless of local signal power.

Comparison of Transversal vs. Lattice Predictor Structures

Property	Transversal (Tapped Delay Line)	Lattice Structure
Modularity	Low (adding a tap changes all weights)	High (adding a tap is just adding a stage)
Stability Check	Complex (requires finding roots of polynomial)	Simple (checks if $ \kappa_m < 1$ for all stages)
Numerical Sensitivity	High (sensitive to coefficient quantization)	Low (robust to finite word-length effects)
Complexity	$O(M)$ multiplications per update	$O(M)$ multiplications per update (slightly higher constant)

Module 5

Introduction to Recursive Least Squares (RLS)

1. Introduction to Recursive Least Squares (RLS)

The Recursive Least Squares (RLS) algorithm minimizes a deterministic, exponentially weighted sum of squared errors, rather than statistical ensemble averages.

1.1 RLS Cost Function

The cost function to minimize at time n is:

$$\mathcal{E}(n) = \sum_{i=1}^n \lambda^{n-i} |e(i, n)|^2 \quad (56)$$

where $\lambda \in (0, 1]$ is the **exponential forgetting factor** (typically $0.95 \leq \lambda \leq 1.0$), and the error signal is $e(i, n) = d(i) - \mathbf{w}^H(n)\mathbf{x}(i)$ for $i = 1, 2, \dots, n$, using the weight vector at current time n .

1.2 Least-Squares Normal Equations

Setting the gradient of $\mathcal{E}(n)$ with respect to $\mathbf{w}^*$ to zero yields the deterministic normal equations:

$$\mathbf{\Phi}(n)\mathbf{w}(n) = \mathbf{z}(n) \implies \mathbf{w}(n) = \mathbf{\Phi}^{-1}(n)\mathbf{z}(n) \quad (57)$$

where the deterministic correlation matrix $\mathbf{\Phi}(n)$ and cross-correlation vector $\mathbf{z}(n)$ are:

$$\mathbf{\Phi}(n) = \sum_{i=1}^n \lambda^{n-i} \mathbf{x}(i)\mathbf{x}^H(i) = \lambda \mathbf{\Phi}(n-1) + \mathbf{x}(n)\mathbf{x}^H(n) \quad (58)$$

$$\mathbf{z}(n) = \sum_{i=1}^n \lambda^{n-i} \mathbf{x}(i)d^*(i) = \lambda \mathbf{z}(n-1) + \mathbf{x}(n)d^*(n) \quad (59)$$

2. Vector Space Formulation of RLS and Matrix Inversion Lemma

2.1 Matrix Inversion Lemma (Woodbury Identity)

To update $\mathbf{\Phi}^{-1}(n)$ without direct inversion, we use the Matrix Inversion Lemma:

$$(\mathbf{A} + \mathbf{BCD})^{-1} = \mathbf{A}^{-1} - \mathbf{A}^{-1}\mathbf{B}(\mathbf{C}^{-1} + \mathbf{DA}^{-1}\mathbf{B})^{-1}\mathbf{DA}^{-1} \quad (60)$$

By defining $\mathbf{P}(n) = \mathbf{P}^{-1}(n)$ and substituting $\mathbf{A} = \lambda \mathbf{P}(n-1)$, $\mathbf{B} = \mathbf{x}(n)$, $\mathbf{C} = 1$, and $\mathbf{D} = \mathbf{x}^H(n)$, we obtain:

$$\mathbf{P}(n) = \lambda^{-1} \mathbf{P}(n-1) - \frac{\lambda^{-2} \mathbf{P}(n-1) \mathbf{x}(n) \mathbf{x}^H(n) \mathbf{P}(n-1)}{1 + \lambda^{-1} \mathbf{x}^H(n) \mathbf{P}(n-1) \mathbf{x}(n)} \quad (61)$$

2.2 The Gain Vector

Defining the gain vector $\mathbf{k}(n)$ as:

$$\mathbf{k}(n) = \frac{\mathbf{P}(n-1) \mathbf{x}(n)}{\lambda + \mathbf{x}^H(n) \mathbf{P}(n-1) \mathbf{x}(n)} = \mathbf{P}(n) \mathbf{x}(n) \quad (62)$$

Allows writing the inverse correlation update compactly as:

$$\mathbf{P}(n) = \lambda^{-1} [\mathbf{P}(n-1) - \mathbf{k}(n) \mathbf{x}^H(n) \mathbf{P}(n-1)] \quad (63)$$

3. RLS Transversal Adaptive Filter Algorithm

The complete step-by-step transversal RLS algorithm is structured as follows:

RLS Algorithm Steps

Initialization (at $n = 0$):

$$\mathbf{w}(0) = \mathbf{0} \quad (64)$$

$$\mathbf{P}(0) = \delta^{-1} \mathbf{I} \quad (\delta \text{ is a small positive constant}) \quad (65)$$

Recursion (for each time index $n = 1, 2, \dots$):

1. Compute the gain vector:

$$\mathbf{k}(n) = \frac{\mathbf{P}(n-1) \mathbf{x}(n)}{\lambda + \mathbf{x}^H(n) \mathbf{P}(n-1) \mathbf{x}(n)}$$

2. Calculate the a priori estimation error:

$$\xi(n) = d(n) - \mathbf{w}^H(n-1) \mathbf{x}(n)$$

3. Update the filter weights:

$$\mathbf{w}(n) = \mathbf{w}(n-1) + \mathbf{k}(n) \xi^*(n)$$

4. Update the inverse correlation matrix:

$$\mathbf{P}(n) = \lambda^{-1} [\mathbf{P}(n-1) - \mathbf{k}(n) \mathbf{x}^H(n) \mathbf{P}(n-1)]$$

3.1 RLS Lattice Filters

RLS lattice filters (Least-Squares Lattice, LSL) combine the fast convergence of least-squares algorithms with the modular, low-computational complexity $O(M)$ of lattice structures, solving numerical sensitivity issues of standard RLS.

4. Advanced Topics

4.1 Affine Projection Algorithm (APA)

APA generalizes NLMS. Instead of adjusting the weight vector based on the current input vector $\mathbf{x}(n)$, APA projects the weights onto a subspace spanned by P past input vectors (where P is the projection order).

- **Significance:** For $P = 1$, APA reduces to NLMS. For $P = M$ (filter length), it approaches RLS performance. It offers a flexible trade-off between computational complexity and convergence speed for highly correlated inputs.

4.2 Subspace-Based Adaptive Filters

These algorithms decompose the signal space into orthogonal signal and noise subspaces using eigendecomposition or singular value decomposition (SVD). By projecting updates exclusively onto the signal subspace, it filters out noise components, enhancing performance in high-resolution estimation tasks.

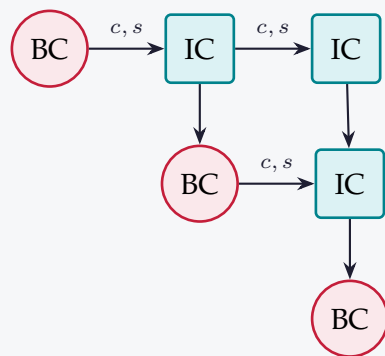
4.3 Partial Update Algorithms

To save power and processor cycles in resource-constrained environments (e.g., mobile devices), these algorithms update only a subset of the M filter coefficients at each iteration, selecting either a fixed subset (Periodic LMS) or a dynamic subset based on magnitude (Max-NLMS).

4.4 QR Decomposition (QRD-RLS) and Systolic Arrays

Standard RLS is numerically unstable under finite word-length effects because the matrix $\mathbf{P}(n)$ can lose its positive-definiteness due to round-off errors.

- **QRD-RLS:** Solves the least-squares problem by applying unitary transformations (Givens rotations) directly to the data matrix $\mathbf{X}(n)$, decomposing it into an orthogonal matrix $\mathbf{Q}(n)$ and an upper triangular matrix $\mathbf{R}(n)$. Since unitary transformations preserve norms, QRD-RLS is numerically stable.
- **Systolic Arrays:** A systolic array is a parallel network of processors (Boundary Cells and Internal Cells) that pipelined Givens rotations for real-time QRD-RLS VLSI hardware execution.

Systolic Array Structure for QRD-RLS

BC: Boundary Cell
(Computes Givens Rotations)
IC: Internal Cell
(Applies Givens Rotations)

Example: RLS Iteration Computation

Given $\lambda = 1.0$, the current estimate $\mathbf{w}(n-1) = [0.5, -0.2]^T$, and the inverse correlation matrix:

$$\mathbf{P}(n-1) = \begin{bmatrix} 2.0 & 0.5 \\ 0.5 & 1.0 \end{bmatrix}$$

The new input sample vector is $\mathbf{x}(n) = [1.0, 2.0]^T$, and the desired signal is $d(n) = 0.8$. Execute one step of the RLS recursion to find the updated weight vector $\mathbf{w}(n)$.

Solution:

1. Compute the numerator of the gain vector:

$$\mathbf{P}(n-1)\mathbf{x}(n) = \begin{bmatrix} 2.0 & 0.5 \\ 0.5 & 1.0 \end{bmatrix} \begin{bmatrix} 1.0 \\ 2.0 \end{bmatrix} = \begin{bmatrix} (2 \times 1) + (0.5 \times 2) \\ (0.5 \times 1) + (1 \times 2) \end{bmatrix} = \begin{bmatrix} 3.0 \\ 2.5 \end{bmatrix}$$

2. Compute the denominator of the gain vector:

$$\lambda + \mathbf{x}^T(n)\mathbf{P}(n-1)\mathbf{x}(n) = 1.0 + \begin{bmatrix} 1.0 & 2.0 \end{bmatrix} \begin{bmatrix} 3.0 \\ 2.5 \end{bmatrix} = 1.0 + (3.0 + 5.0) = 9.0$$

3. Compute the gain vector $\mathbf{k}(n)$:

$$\mathbf{k}(n) = \frac{1}{9.0} \begin{bmatrix} 3.0 \\ 2.5 \end{bmatrix} = \begin{bmatrix} 1/3 \\ 5/18 \end{bmatrix} \approx \begin{bmatrix} 0.3333 \\ 0.2778 \end{bmatrix}$$

4. Compute the a priori estimation error $\xi(n)$:

$$\xi(n) = d(n) - \mathbf{w}^T(n-1)\mathbf{x}(n) = 0.8 - \begin{bmatrix} 0.5 & -0.2 \end{bmatrix} \begin{bmatrix} 1.0 \\ 2.0 \end{bmatrix} = 0.8 - (0.5 - 0.4) = 0.7$$

5. Update the weight vector $\mathbf{w}(n)$:

$$\mathbf{w}(n) = \mathbf{w}(n-1) + \mathbf{k}(n)\xi(n) = \begin{bmatrix} 0.5 \\ -0.2 \end{bmatrix} + 0.7 \begin{bmatrix} 1/3 \\ 5/18 \end{bmatrix} = \begin{bmatrix} 0.5 + 0.2333 \\ -0.2 + 0.1944 \end{bmatrix} = \begin{bmatrix} 0.7333 \\ -0.0056 \end{bmatrix}$$

Quick Revision — Module V

Recursive Least Squares (RLS) & Advanced Topics | PE-EC702A

Key Formulas at a Glance

RLS Cost Function	$\mathcal{E}(n) = \sum_{i=1}^n \lambda^{n-i} e(i, n) ^2$
Correlation Matrix Update	$\mathbf{P}(n) = \lambda \mathbf{P}(n-1) + \mathbf{x}(n)\mathbf{x}^H(n)$
Matrix Inversion Lemma	$(\mathbf{A} + \mathbf{BCD})^{-1} = \mathbf{A}^{-1} - \mathbf{A}^{-1}\mathbf{B}(\mathbf{C}^{-1} + \mathbf{DA}^{-1}\mathbf{B})^{-1}\mathbf{DA}^{-1}$
Gain Vector $\mathbf{k}(n)$	$\mathbf{k}(n) = \frac{\mathbf{P}(n-1)\mathbf{x}(n)}{\lambda + \mathbf{x}^H(n)\mathbf{P}(n-1)\mathbf{x}(n)}$
Weight Update	$\mathbf{w}(n) = \mathbf{w}(n-1) + \mathbf{k}(n)\xi^*(n) \quad (\xi(n) = d(n) - \mathbf{w}^H(n-1)\mathbf{x}(n))$
Inverse Matrix Update	$\mathbf{P}(n) = \lambda^{-1} [\mathbf{P}(n-1) - \mathbf{k}(n)\mathbf{x}^H(n)\mathbf{P}(n-1)]$

One-Line Definitions

- **RLS Algorithm:** An adaptive filter algorithm that recursively finds coefficients to minimize deterministic sum of squared errors.
- **Forgetting Factor:** A scalar parameter $\lambda \leq 1.0$ weighting recent data samples more heavily than older ones.
- **Gain Vector:** The vector $\mathbf{k}(n)$ that scales the error term in the RLS weight update equation.
- **Matrix Inversion Lemma:** A mathematical formula allowing recursive computation of matrix inverses without direct inversion.
- **Systolic Array:** A network of processing elements configured to execute parallel QRD matrix operations in hardware.
- **Boundary Cell:** A processor in a QRD systolic array calculating Givens rotation parameters (cosines and sines).

Frequently Asked / Viva Questions

Q1. Compare RLS and LMS in terms of convergence speed and computational complexity.

RLS converges much faster than LMS (typically within $2M$ iterations, independent of eigenvalue spread) but requires higher computational complexity ($O(M^2)$ multiplications per sample compared to $O(M)$ for LMS).

Q2. Explain the role of the forgetting factor λ in the RLS cost function.

λ determines the memory of the filter. If $\lambda = 1.0$, the filter has infinite memory (all data weighted equally). If $\lambda < 1.0$ (typically 0.98), older data is exponentially

Frequently Asked / Viva Questions (contd.)

attenuated, allowing the filter to track non-stationary changes.

Q3. Write down the Woodbury identity (Matrix Inversion Lemma) and explain its use in RLS.

$(\mathbf{A} + \mathbf{UCV})^{-1} = \mathbf{A}^{-1} - \mathbf{A}^{-1}\mathbf{U}(\mathbf{C}^{-1} + \mathbf{VA}^{-1}\mathbf{U})^{-1}\mathbf{VA}^{-1}$. It is used to recursively update the inverse correlation matrix $\mathbf{P}(n) = \mathbf{\Lambda}^{-1}(n)$ without performing an $O(M^3)$ matrix inversion step.

Q4. What are the typical initialization values for the RLS algorithm?

$\mathbf{w}(0) = \mathbf{0}$ and $\mathbf{P}(0) = \delta^{-1}\mathbf{I}$, where δ is a small positive regularization parameter (e.g., 0.01).

Q5. What is the difference between a priori error $\xi(n)$ and a posteriori error $e(n)$ in RLS?

The a priori error $\xi(n) = d(n) - \mathbf{w}^H(n-1)\mathbf{x}(n)$ is calculated using the old weight vector, while the a posteriori error $e(n) = d(n) - \mathbf{w}^H(n)\mathbf{x}(n)$ is calculated using the updated weight vector.

Q6. What is the purpose of using QR Decomposition (QRD) in RLS filters?

Standard RLS suffers from numerical instability under finite precision arithmetic, causing $\mathbf{P}(n)$ to lose positive-definiteness. QRD solves this by applying unitary rotations to the data matrix, ensuring numerical stability.

Q7. What is the Affine Projection Algorithm (APA)?

APA is an intermediate algorithm between NLMS and RLS that updates weights based on the current and $P - 1$ past input vectors, speeding up convergence for correlated signals with moderate complexity.

Q8. What are partial update algorithms and where are they useful?

They are algorithms that update only a fraction of the filter weights at each time index to reduce power consumption and processing requirements, useful in acoustic echo cancellation in mobile phones.

Q9. What are the functions of boundary cells and internal cells in a systolic array?

Boundary cells compute the Givens rotation parameters (cosine c and sine s) to zero out elements of the incoming data matrix, while internal cells apply these rotations to the rest of the matrix rows.

Q10. Why does RLS converge independently of the eigenvalue spread of the input signal?

Because RLS pre-whitens the input signal by multiplying the error vector with the inverse correlation matrix $\mathbf{P}(n)$, which de-correlates the input modes.

Comparison of LMS vs. RLS Algorithms

Parameter	LMS Algorithm	RLS Algorithm
Convergence Rate	Slow (dependent on input eigenvalue spread)	Fast (typically within $2M$ iterations, independent of spread)
Complexity	$O(M)$ multiplications per sample	$O(M^2)$ multiplications per sample
Stability	Stable if $0 < \mu < 2/\text{Tr}(\mathbf{R})$	Can suffer from numerical instability in finite precision
Parameters	Step-size μ	Forgetting factor λ , regularization δ